

CTF TINGKAT LANJUT – CRYPTOGRAPHY (PYTHON BASED)

SOAL 1 — RSA Common Modulus Attack

Deskripsi Challenge

Diberikan:

```
# public values
n = 119315717514047
e1 = 65537
e2 = 17

c1 = 104085...
c2 = 948573...
```

Kedua ciphertext berasal dari plaintext yang sama, tetapi dienkripsi dengan dua exponent berbeda dan modulus yang sama.

Tugas:

Recover plaintext dan temukan flag dalam format:

flag{plaintext}

Konsep

Jika:

$$c1 = m^{e1} \bmod n$$

$$c2 = m^{e2} \bmod n$$

Dan $\gcd(e1, e2) = 1$, maka bisa digunakan:

Extended Euclidean Algorithm

Untuk mencari:

$$a \cdot e1 + b \cdot e2 = 1$$

Sehingga:

$$m = (c1^a \cdot c2^b) \bmod n$$

Pembahasan (Solve Script)

```
from Crypto.Util.number import long_to_bytes
from math import gcd
```

```

# Extended Euclid
def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    g, y, x = egcd(b % a, a)
    return (g, x - (b // a) * y, y)

n = 119315717514047
e1 = 65537
e2 = 17
c1 = ...
c2 = ...

g, a, b = egcd(e1, e2)

m = (pow(c1, a, n) * pow(c2, b, n)) % n
print(long_to_bytes(m))

```

SOAL 2 — AES ECB Detection & Decryption

Challenge

Diberikan file terenkripsi AES:

```
ciphertext = open("encrypted.bin", "rb").read()
```

Hint:

- Mode yang digunakan adalah AES
- Kunci adalah 16 byte dan diketahui sebagian: b"SECRET_KEY_????"

Tugas:

1. Deteksi mode AES
2. Brute-force 4 digit terakhir key
3. Recover flag

Konsep

Mode ECB memiliki ciri:

Block ciphertext yang identik → plaintext block identik

Pembahasan

```
from Crypto.Cipher import AES
from itertools import product

ciphertext = open("encrypted.bin", "rb").read()

for digits in product("0123456789", repeat=4):
    key = b"SECRET_KEY_" + "".join(digits).encode()
    cipher = AES.new(key, AES.MODE_ECB)
    try:
        plaintext = cipher.decrypt(ciphertext)
        if b"flag{" in plaintext:
            print("FOUND:", key)
            print(plaintext)
            break
    except:
        pass
```

SOAL 3 — XOR Known Plaintext Attack

Challenge

Diberikan:

```
cipher = bytes.fromhex("3f2a1b4c5d...")
```

Diketahui:

- Format flag selalu: flag{...}
- XOR dengan repeating key

Tugas:

Recover key dan flag.

Konsep

Jika:

```
cipher = plaintext XOR key
```

Dan sebagian plaintext diketahui, maka:

```
key = cipher XOR known_plaintext
```

Pembahasan

```
cipher = bytes.fromhex("3f2a1b4c5d...")
known = b"flag{"
key = bytes([cipher[i] ^ known[i] for i in range(len(known))])
print("Recovered partial key:", key)

plaintext = bytes([cipher[i] ^ key[i % len(key)] for i in range(len(cipher))])
print(plaintext)
```

SOAL 4 — ECDSA Nonce Reuse Attack

Challenge

Diberikan dua signature ECDSA dengan nonce yang sama:

(r, s1)

(r, s2)

Dengan:

$z1 = \text{hash}(m1)$

$z2 = \text{hash}(m2)$

Tugas:

Recover private key.

Konsep

Jika nonce k reused:

$$s1 = k^{-1}(z1 + r*d)$$
$$s2 = k^{-1}(z2 + r*d)$$

Maka:

$$k = (z1 - z2) / (s1 - s2)$$

$$d = (s1 * k - z1) / r$$

Pembahasan

```
from Crypto.Util.number import inverse
```

```
n = curve_order
```

```
k = ((z1 - z2) * inverse(s1 - s2, n)) % n
```

```
d = ((s1 * k - z1) * inverse(r, n)) % n
```

```
print("Private key:", d)
```

SOAL 5 — Padding Oracle Attack (Advanced)

Challenge

Server memberi respon:

- "Valid padding"
- "Invalid padding"

AES-CBC digunakan.

Tugas:

Decrypt ciphertext tanpa mengetahui key.

Konsep

CBC decryption:

$$P_i = D(C_i) \text{ XOR } C_{i-1}$$

Dengan memodifikasi C_{i-1} , kita bisa brute-force byte padding.

Contoh Exploit Script

```
import requests
```

```
def oracle(cipher):
```

```
    r = requests.post("http://target.com/decrypt", data=cipher.hex())
```

```
    return "Valid" in r.text
```

```

block_size = 16
recovered = bytearray(block_size)

for i in range(1, block_size+1):
    for guess in range(256):
        modified = bytearray(original_block)
        modified[-i] ^= guess ^ i

        if oracle(modified + target_block):
            recovered[-i] = guess
            break

print("Recovered block:", recovered)

```

Ringkasan Tingkat Kesulitan

No	Topik	Tingkat
1	RSA Common Modulus	★★★★
2	AES ECB Brute Force	★★★
3	XOR Attack	★★
4	ECDSA Nonce Reuse	★★★★★
5	Padding Oracle	★★★★★

PAKET CTF CRYPTOGRAPHY – VERSI LOMBA INTERNAL SEKOLAH

FORMAT LOMBA

Nama Event : CryptoWar School CTF
Durasi : 3 Jam
Format Flag : CTF{...}
Kategori : Cryptography
Jumlah Soal : 5
Total Poin : 500

CHALLENGE 1 — “XOR is Not Secure”

Kategori : Crypto
Poin : 50
Level : Easy

Deskripsi Soal

Diberikan file encrypted.txt:

```
1f0a071c09453b0a0d...
```

Hint:

“Same key. Again and again.”

Tujuan

Recover flag.

Konsep

Repeating XOR key.

File Generator (Untuk Panitia)

```
key = b"school"
flag = b"CTF{XOR_IS_WEAK}"
cipher = bytes([flag[i] ^ key[i % len(key)] for i in range(len(flag))])

print(cipher.hex())
```

Solver (Contoh Jawaban)

```
cipher = bytes.fromhex("...")

for key_length in range(2,10):
    key = cipher[:key_length]
    plaintext = bytes([cipher[i] ^ key[i % key_length] for i in range(len(cipher))])
    if b"CTF{" in plaintext:
        print(plaintext)
```

CHALLENGE 2 — “ECB Penguin”

Kategori : Crypto

Poin : 100

Level : Medium

Deskripsi

Diberikan file encrypted.bin

Hint:

“Block by block, I reveal myself.”

Tujuan

1. Identifikasi mode
2. Brute force 4 digit PIN key

Generator Soal

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

key = b"SCHOOL_KEY_1234"
cipher = AES.new(key, AES.MODE_ECB)

flag = b"CTF{ECB_MODE_IS_BAD}"
ct = cipher.encrypt(pad(flag,16))

open("encrypted.bin","wb").write(ct)
```


Expected Solve

Brute-force 0000–9999 pada 4 digit terakhir.

CHALLENGE 3 — “Broken RSA”

Kategori : Crypto

Poin : 150

Level : Medium-Advanced

Deskripsi

Diberikan:

```
n
e1
e2
c1
c2
Plaintext sama.
```

Tujuan

Gunakan Common Modulus Attack.

Generator

```
from Crypto.Util.number import *
p = getPrime(64)
q = getPrime(64)
n = p*q

e1 = 65537
e2 = 17

flag = b"CTF{COMMON_MODULUS_ATTACK}"
m = bytes_to_long(flag)

c1 = pow(m,e1,n)
c2 = pow(m,e2,n)
```

```
print(n,e1,e2,c1,c2)
```

Solve Core

```
from Crypto.Util.number import inverse, long_to_bytes
```

```
def egcd(a,b):
```

```
    if a==0:
```

```
        return (b,0,1)
```

```
    g,y,x = egcd(b%a,a)
```

```
    return (g,x-(b//a)*y,y)
```

```
g,a,b = egcd(e1,e2)
```

```
m = (pow(c1,a,n)*pow(c2,b,n))%n
```

```
print(long_to_bytes(m))
```

CHALLENGE 4 — “Nonce Reuse Disaster”

Kategori : Crypto

Poin : 100

Level : Advanced

Deskripsi

Diberikan dua signature ECDSA dengan r yang sama.

Hint:

“Never reuse randomness.”

Tujuan

Recover private key dan flag.

Generator

```
# Simulasi nilai saja
```

```
d = 123456789
```

```
flag = f"CTF{{PRIVATE_{{d}}}}"
```

Panitia hanya memberikan:

r
s1
s2
z1
z2
n

Solve

Gunakan rumus:

$$k = (z1 - z2)/(s1 - s2)$$

$$d = (s1*k - z1)/r$$

CHALLENGE 5 — “Padding Oracle Server”

Kategori : Crypto

Poin : 100

Level : Advanced

Deskripsi

Disediakan server:

<http://localhost:5000/decrypt>

Server memberi respon:

- Valid padding
- Invalid padding

Server Challenge (Untuk Panitia)

```
from flask import Flask, request
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import os
```

```
app = Flask(__name__)
```

```
key = os.urandom(16)
```

```
cipher = AES.new(key, AES.MODE_CBC)
flag = pad(b"CTF{PADDING_ORACLE_ATTACK}",16)
```

```
@app.route("/decrypt", methods=["POST"])
```

```
def decrypt():
```

```
    data = bytes.fromhex(request.data.decode())
```

```
    cipher = AES.new(key, AES.MODE_CBC, data[:16])
```

```
    try:
```

```
        unpad(cipher.decrypt(data[16:]),16)
```

```
        return "Valid padding"
```

```
    except:
```

```
        return "Invalid padding"
```

```
app.run()
```

SCORING SYSTEM

Soal	Poin
XOR	50
AES ECB	100
RSA	150
ECDSA	100
Padding Oracle	100

Total: 500 poin

Mekanisme Penilaian

Opsi 1 – Manual

Peserta submit flag → panitia cek.

Opsi 2 – Platform Lokal

Gunakan:

- CTFd
- FBCTF
- PicoCTF

Infrastruktur Minimal

- 1 VPS / Server lokal
- Python 3.10+
- Library:
 - ✓ pycryptodome
 - ✓ flask

Skill yang Diuji

- Analisis matematis RSA
- Block cipher attack
- Signature vulnerability
- XOR analysis
- Web crypto exploit

